

MediciMess — Tech Stack Overview & Setup Guide

Tech stack overview for the MediciMess forensic data analysis project (embezzlement detection on the Medici Bank transaction dataset).

Full stack:

- Python 3.11+
- pandas — ingestion and transformations
- `Decimal` (or carefully controlled two-decimal float handling) — financial calculations
- FastAPI — REST API
- SQLite or DuckDB — local storage/querying
- Streamlit — app/dashboard front end
- Plotly — charts
- pytest — unit and integration testing
- Ruff or Black — formatting

1. SQLite3 vs. DuckDB — which to use

	SQLite3	DuckDB
Type	Row-oriented (OLTP)	Column-oriented (OLAP)
Best at	Many small reads/writes, transactional data	Aggregations, analytics, scanning large columns fast
Concurrency	Single writer at a time	Single writer at a time (similar model)
Setup	Built into Python stdlib	<code>pip install duckdb</code>
File format	<code>.db</code> / <code>.sqlite</code>	<code>.duckdb</code> (or query files directly, no import)
Can read the other's files?	No	Yes — DuckDB can query SQLite files and CSV/Parquet directly

For MediciMess specifically: the raw data lives in `medici_transactions.csv/.json` (80,000+ rows) — DuckDB is the natural fit for the analysis side since it queries CSVs directly with no import step and is built for the kind of aggregation/grouping forensic analysis needs (vendor concentration, Benford's Law digit distributions, etc.). SQLite is the better fit if/when the FastAPI backend needs a simple transactional store (e.g. flagged-transaction records, alert history for the dashboard spec).

2. FastAPI Refresher

Install

```
pip install fastapi uvicorn[standard]
```

Minimal app

```
# main.py
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.get("/")
def read_root():
    return {"status": "ok"}
```

Run it:

```
uvicorn main:app --reload
```

Auto docs land at <http://127.0.0.1:8000/docs>.

Request/response validation with Pydantic

```
from pydantic import BaseModel
```

```
class Item(BaseModel):
    name: str
    price: float
```

```
@app.post("/items")
def create_item(item: Item):
    return {"received": item}
```

Dependency injection (this is how we'll hand out DB connections)

```
from fastapi import Depends
```

```
def get_db():
    db = connect_to_db()
    try:
        yield db
    finally:
        db.close()
```

```
@app.get("/items")
```

```
def list_items(db=Depends(get_db)):
    return db.query(...)
```

`Depends()` runs `get_db()` before the endpoint, injects whatever it `yields`, and runs the cleanup after the response is sent. Same pattern for both SQLite3 and DuckDB connections below.

3. SQLite3 Setup

No install needed — it's in the Python standard library.

Connect and create a table

```
import sqlite3

conn = sqlite3.connect("app.db")
cur = conn.cursor()
cur.execute("""
    CREATE TABLE IF NOT EXISTS flagged_transactions (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        transaction_id TEXT NOT NULL,
        reason TEXT,
        flagged_at TEXT DEFAULT CURRENT_TIMESTAMP
    )
""")
conn.commit()
```

Basic CRUD

```
# Insert
cur.execute("INSERT INTO flagged_transactions
(transaction_id, reason) VALUES (?, ?)", ("TX10432",
"Vendor concentration outlier"))
conn.commit()

# Read
cur.execute("SELECT * FROM flagged_transactions")
rows = cur.fetchall()

# Update
cur.execute("UPDATE flagged_transactions SET reason = ?
WHERE id = ?", ("Confirmed embezzlement pattern", 1))
```

```
conn.commit()
```

```
# Delete
```

```
cur.execute("DELETE FROM flagged_transactions WHERE id  
= ?", (1,))
```

```
conn.commit()
```

Always use ? placeholders (parameterized queries) instead of f-strings — avoids SQL injection.

FastAPI wiring

```
import sqlite3
```

```
from fastapi import FastAPI, Depends
```

```
app = FastAPI()
```

```
def get_db():
```

```
    conn = sqlite3.connect("app.db")
```

```
    conn.row_factory = sqlite3.Row # lets you access  
columns by name
```

```
    try:
```

```
        yield conn
```

```
    finally:
```

```
        conn.close()
```

```
@app.get("/flagged")
```

```
def list_flagged(db: sqlite3.Connection = Depends(get_db)):
```

```
    rows = db.execute("SELECT * FROM  
flagged_transactions").fetchall()
```

```
    return [dict(row) for row in rows]
```

4. DuckDB Setup

Install

```
pip install duckdb
```

Query the transaction CSV directly — no import step

```
import duckdb
```

```
con = duckdb.connect("medici_analytics.duckdb")
```

```
con.execute("""
    CREATE TABLE IF NOT EXISTS transactions AS
    SELECT * FROM 'medici_transactions.csv'
""")
```

Querying — same SQL you already know

```
result = con.execute("SELECT * FROM transactions LIMIT
5").fetchall()
```

Or straight to a dataframe (handy for pandas-based analysis)

```
df = con.execute("SELECT * FROM transactions").df()
```

Example: vendor concentration query (relevant to the forensic analysis)

```
con.execute("""
    SELECT credit_account, COUNT(*) AS tx_count,
    SUM(CAST(credit_amount AS DOUBLE)) AS total
    FROM transactions
    WHERE branch = 'Florence' AND date BETWEEN '1420-01-01'
    AND '1424-12-31'
    GROUP BY credit_account
    ORDER BY total DESC
""").df()
```

FastAPI wiring

```
import duckdb
from fastapi import FastAPI, Depends

app = FastAPI()

def get_analytics_db():
    con = duckdb.connect("medici_analytics.duckdb")
    try:
        yield con
    finally:
        con.close()

@app.get("/analytics/vendor-concentration")
def vendor_concentration(con=Depends(get_analytics_db)):
    result = con.execute(
```

```

        "SELECT credit_account, SUM(CAST(credit_amount AS
DOUBLE)) AS total FROM transactions GROUP BY credit_account
ORDER BY total DESC"
    ).fetchall()
    return result

```

5. SQLite3 ↔ DuckDB — bridging, if needed

If flagged transactions end up in SQLite (Section 3) and you want to join them against the DuckDB analytics table, DuckDB can attach and query SQLite directly:

```

con.execute("INSTALL sqlite; LOAD sqlite;")
con.execute("ATTACH 'app.db' AS app (TYPE sqlite)")
con.execute("SELECT * FROM app.flagged_transactions").df()

```

No separate ETL step needed — DuckDB reads the live SQLite file in place.

6. Decimal / Financial Precision

Floats lose precision in ways that matter for accounting — $0.1 + 0.2 \neq 0.3$ in binary float. For anything touching debits/credits/balances, use `decimal.Decimal`:

```

from decimal import Decimal, ROUND_HALF_UP

amount = Decimal("1500.00")
interest = amount * Decimal("0.02")
rounded = interest.quantize(Decimal("0.01"),
rounding=ROUND_HALF_UP)

```

Notes for this project:

- The dataset's CSV/JSON amounts come in as strings — parse them straight into `Decimal(str(value))`, never `float(value)` first (that reintroduces the precision loss before you can fix it).
- If a step needs pandas/DuckDB for speed (bulk aggregation), it's reasonable to work in floats there and only drop into `Decimal` for the final reported/validated figures — just be explicit about where that boundary is so debit/credit checks stay exact.

7. Streamlit + Plotly

Streamlit is the front end for the dashboard; Plotly supplies the charts.

Install

```
pip install streamlit plotly
```

Minimal app

```
# dashboard.py
import streamlit as st
import plotly.express as px
import pandas as pd

st.title("MediciMess – Branch Operations Dashboard")

df = pd.read_csv("medici_transactions.csv")
fig = px.line(df, x="date", y="credit_amount",
color="branch")
st.plotly_chart(fig, use_container_width=True)
Run it:
```

```
streamlit run dashboard.py
```

This is the natural place to implement the KPI panels and anomaly/alert views called for in `BRANCH_OPS_UI_SPEC.md`.

8. pytest

Install

```
pip install pytest
```

Example test structure

```
# tests/test_ledger.py
from decimal import Decimal
from medici_banking import Ledger, AccountType,
TransactionEntry

def test_transaction_balances():
    ledger = Ledger("Test Bank")
    cash = ledger.create_account("Cash", AccountType.ASSET)
    capital = ledger.create_account("Capital",
AccountType.EQUITY)
```

```

txn = ledger.record_transaction(
    "2024-01-01", "Initial investment",
    TransactionEntry(cash, Decimal("100.00")),
    TransactionEntry(capital, Decimal("100.00")),
)
assert txn.is_balanced()

```

Run it:

`pytest`

Worth having tests for: transaction balance validation, import/export round-tripping (CSV/JSON), and any anomaly-detection function (Benford's Law, vendor concentration) against known-good and known-flagged sample data.

9. Ruff / Black (formatting)

Pick one, apply consistently across the team.

```

pip install ruff
ruff check .
ruff format .
or

```

```

pip install black
black .

```

10. Suggested Project Structure

```

medicimess/
├── main.py                # FastAPI app + routes
├── dashboard.py           # Streamlit app
├── db/
│   └── sqlite.py          # SQLite connection +
get_db dependency
│   └── duckdb_conn.py     # DuckDB connection +
get_analytics_db dependency
├── analysis/
│   └── forensics.py       # Benford's Law, vendor
concentration, frequency analysis
└── notebooks/

```



```

├── data_exploration.ipynb  # Exploration notebook
├── tests/
│   └── test_*.py
├── medici_transactions.csv
├── medici_analytics.duckdb
├── app.db
├── pyproject.toml          # Ruff/Black config
├── requirements.txt
requirements.txt:

```

```

fastapi
uvicorn[standard]
duckdb
pandas
streamlit
plotly
pytest
ruff
(sqlite3 and decimal need no entry — both stdlib.)

```

11. Next Steps for the Team

1. Everyone installs dependencies: `pip install -r requirements.txt`
2. Confirm which storage each piece uses: SQLite for flagged-transaction/alert records, DuckDB for bulk analytics queries against the CSV
3. Standardize on `Decimal` for any debit/credit/balance math — float only for exploratory aggregation
4. Data exploration notebook (in progress) feeds the anomaly-detection functions in `analysis/forensics.py`
5. Streamlit dashboard consumes the FastAPI analytics endpoints once they're stable
6. Pick Ruff or Black and add the config to `pyproject.toml` so formatting is consistent across the team